BROCCOLI BOARD LAB

Phased Build Guide

Custom FC+ESC Drone — Component by Component, Item by Item

Revised: freestyle-capable ESC · custom 3D-printed frame · transmitter included

Prepared for broccoli ■ — July 2026

Index

How This Guide Works

Each component is one **phase**. Each phase has numbered **items** — the exact supporting parts (capacitors, resistors, ferrite beads, crystals) that go with that component. Finish every item in a phase, double-check it against the datasheet reference, then move to the next phase. Don't jump ahead — if something breaks later, you want to know it's isolated to the most recent phase, not spread across five half-finished ones.

Build order: MCU → IMU → ESC channel (repeated 4x, one per motor) → Power → Peripherals. This order is deliberate — the MCU's pin assignments have to be locked before anything else can be wired to them.

PHASE 1: Microcontroller — STM32F405RGT6

LQFP-64 · the flight computer · lock every pin assignment here before moving on

Item 1: Power pins

• 100nF ceramic decoupling capacitor on every single VDD pin — do not skip any.

• 1x 4.7–10µF bulk capacitor on the main VDD rail.

Item 2: Analog reference

• 100nF + 1µF on VDDA/VREF+, isolated from digital VDD with a small ferrite bead.

Item 3: Backup rail

• 10nF + 1µF on VBAT, even without a backup RTC battery.

Item 4: Clock

• 1x 8MHz HSE crystal (3225 SMD).

• 2x load capacitors matched to the crystal's specified load capacitance (commonly 10–20pF each).

Item 5: Programming and reset

• 4-pin SWD header: SWDIO, SWCLK, GND, 3V3.

• BOOT0 pushbutton or jumper to enter DFU flashing mode.

• NRST pushbutton with 100nF capacitor to GND for debounce.

Before moving on: write down which physical pins you assigned to SPI (for the IMU), which timer channels you assigned to each motor (for the ESCs), and which UART you assigned to the receiver. Every later phase depends on this list.

Datasheet: STM32F405RGT6, STMicroelectronics — https://www.lcsc.com/datasheet/C15742.pdf

PHASE 2: IMU — ICM-20602

3x3mm QFN · gyro + accelerometer · most noise-sensitive part on the board

Item 1: Power decoupling

• 100nF ceramic on VDD, placed as close to the pin as physically possible.

• 100nF ceramic on VDDIO if IO voltage differs from core VDD.

Item 2: SPI wiring

• SCLK, MOSI, MISO, CS to the SPI pins you assigned in Phase 1.

• Keep these traces short; route away from any motor/ESC power traces.

Datasheet: ICM-20602, TDK InvenSense — https://invensense.tdk.com/wp-content/uploads/2016/10/DS-000176-ICM-20602-v1.0.pdf

Phase 3: ESC Channel Template (build once, repeat x4)

PHASE 3: ESC Channel Template

AT32F421 + FD6288Q + 6x MOSFET · build this block once, then repeat it identically 3 more times, one per motor

This replaces a single integrated driver chip with the real architecture used in freestyle-capable ESCs: a dedicated small MCU per motor, running AM32 firmware, driving a 3-phase gate driver, driving 6 discrete MOSFETs. A fully open-source reference design exists for this exact combination — **incutec-hw/OpenESC_20X20** on GitHub — KiCad source, 20x20mm, 30A/channel, AT32F421 + AM32, Betaflight-compatible. Use it to cross-check every value below.

Item 1: Per-channel MCU — AT32F421G8U7

• 100nF decoupling capacitor on VDD.

• 100nF + 1µF on VDDA.

• SWD header (SWDIO, SWCLK, GND, 3V3) for flashing the AM32 bootloader — needed once per channel during assembly.

• Single-wire signal connection back to the main STM32 (this carries DShot).

Item 2: Gate driver — FD6288Q

• 3x bootstrap capacitors (one per half-bridge), value per the OpenESC_20X20 reference — typically in the 100nF–1µF range for this driver.

• 100nF decoupling on VCC.

• Gate resistors between each driver output and its MOSFET gate, sized to control switching speed and ringing — verify exact value against the reference design rather than guessing.

Item 3: Power stage — 6x N-channel MOSFETs (3 half-bridges)

• Choose MOSFETs with continuous drain current at least double your expected per-motor current draw — pulsed switching is harder on a MOSFET than steady DC.

• Confirm exact part number and footprint from the OpenESC_20X20 BOM before ordering — this is the one value worth copying exactly rather than approximating.

• 1x bulk low-ESR, high-ripple-current capacitor per channel on the motor supply rail (rule of thumb from the AM32 hardware design guide: undersizing this causes ESCs to fail under rapid current swings).

Item 4: Motor output

• 3 phase pads per channel, routed to a connector or direct solder pad for the motor's 3 wires.

Repeat this entire phase 3 more times — once per motor — before moving to Phase 4.

Reference hardware (KiCad, open source): https://github.com/incutec-hw/OpenESC_20X20

AT32F421 datasheet: https://www.lcsc.com/datasheet/lcsc_datasheet_2206061416_ARTERY-AT32F421G8U7_C2765098.pdf

FD6288Q datasheet: https://lcsc.com/datasheet/lcsc_datasheet_2411011654_Fortior-Tech-FD6288Q_C328453.pdf

AM32 hardware design guide: https://wiki.am32.ca/development/Hardware-Design.html

PHASE 4: Power — Buck (5V) + LDO (3.3V)

Battery to 5V to 3.3V · build last, now that real current draw across all 4 ESC channels is known

Item 1: Input protection

• 1x Schottky diode (≥40V, ≥3A) in series with battery positive for reverse-polarity protection.

• 1x bulk capacitor (100–220µF, ≥25V) across the battery input.

Item 2: Buck regulator — TPS5430

• 10µF input ceramic capacitor near VIN.

• Inductor, 10–33µH, rated for at least 3A saturation — use TI's WEBENCH tool for your exact values.

• Schottky catch diode (≥40V, 3A) — required, this is a non-synchronous buck.

• ~220µF output capacitance (ceramic + electrolytic).

• Feedback resistor divider set for 5V output.

Item 3: LDO — AMS1117-3.3

• 10µF input capacitor.

• 10µF tantalum output capacitor (needs some ESR for stability — not pure ceramic).

Datasheet TPS5430: https://www.ti.com/lit/ds/symlink/tps5430.pdf

Datasheet AMS1117: http://www.advanced-monolithic.com/pdf/ds1117.pdf

Phase 5: Peripherals (optional)

PHASE 5: Peripherals (Optional)

OSD, barometer, current sense, USB-C, receiver header — build any/all last

Item 1: USB-C flashing port

• 2x 5.1kΩ pull-down resistors on CC1/CC2.

• D+/D– direct to the STM32's USB OTG FS pins.

Item 2: ELRS receiver header

• 3–4 pin header: power, GND, TX, RX to a spare UART.

• No extra passives needed — the receiver module has its own regulation.

Item 3: OSD — AT7456E (skip if not running FPV video)

• 100nF on digital VDD; 100nF + 10µF on analog video supply.

• Separate SPI CS line from the IMU.

Item 4: Barometer — BMP280 (optional)

• 100nF on VDDIO and VDD; I2C with 4.7–10k pull-ups.

Item 5: Current sense — INA240 (optional)

• Shunt resistor in the main battery path; 100nF on VS; output to an MCU ADC pin.

Designing the Custom 3D-Printed Frame

This is a mechanical design problem, not an electronics one — the two failure modes to design around are propeller strikes (wrong clearance) and vibration reaching the IMU (arms too flexible).

Decision	Recommendation	Why
Material	PETG or Nylon	PLA is too brittle — cracks on hard landings instead of flexing
Wheelbase	120–150mm motor-to-motor (diagonal)	Must clear 3" (76mm) prop diameter with margin at full lean angle
Arm print orientation	Flat, layer lines running lengthwise	Vertical layers delaminate under vibration — the #1 cause of 3D-printed frame failure
Arm construction	Bolt-on, replaceable per arm	A crash breaks one cheap arm instead of the whole frame
PCB mounting	M3 standoffs on the 20x20mm pattern + soft rubber grommets	Grommets isolate motor vibration before it reaches the gyro
Motor bolt pattern	Confirm exact pattern (commonly M2, 9x12mm) from your specific motor before finalizing arm-end geometry	Wrong hole spacing means the motor simply doesn't bolt on

Design process:

• Model the center plate first — it carries the PCB mounting holes (20x20mm) and defines where arms attach.

• Model one arm, with the motor mounting pattern at the far end and a bolt-on interface to the center plate.

• Pattern the arm 4 times around the center plate at your chosen wheelbase.

• Check prop clearance in CAD with the actual prop diameter modeled — not just eyeballed.

• Print one arm first as a test piece before committing to a full set — verify motor bolt pattern and clearance physically before printing the whole frame.

• Use Fusion 360 (same tool used for the Broccoli Board case) — same workflow, new geometry problem.

Appendix — Updated Bill of Materials

Ordering reference only — build using the phases above.

Qty	Part	Approx. Price	Notes
1	STM32F405RGT6	$4.50	Main flight MCU
1	ICM-20602	$2.20	IMU
4	AT32F421G8U7	$4.00 (4x)	One ESC MCU per motor, runs AM32
4	FD6288Q	$1.60 (4x)	One gate driver per motor
24	N-channel MOSFETs	$14.00 (24x)	6 per motor channel — confirm exact part vs. OpenESC_20X20 BOM
1	TPS5430	$1.50	5V buck
1	AMS1117-3.3	$0.10	3.3V LDO
4	Brushless motors (3" class, e.g. 1404–1606, ~3800–4500KV)	$24.00 (4x)	Sized for real freestyle current draw, not just hover
1 set	3" props (2x CW, 2x CCW)	$6.00	Keep spares — you will break some
1	4S LiPo battery + charger	$25.00	Matches motor KV choice
1	Happymodel ELRS Nano RX	$12.00	Bought module, UART to main MCU
1	RadioMaster Zorro (ELRS)	$70–100	Handheld transmitter — not optional, was missing from original budget
1 set (5pcs)	PCB fabrication (4-layer)	$15–20	JLCPCB or similar
—	Frame filament (PETG/Nylon)	$8–12	Custom printed frame
—	Passives (caps/resistors/diodes)	~$8.00	Across all phases

Estimated grand total: ~$250–300, all-in — board, motors, frame material, battery, receiver, and transmitter. This is the honest cost of a board built for real maneuvers, not just hover.